

Neural Networks

CSE427: Machine Learning

Lecture Roadmap

- 1 Why neural networks?
- 2 Artificial neuron model
- 3 Feedforward vs recurrent neural networks
- 4 Perceptron as a linear classifier
- 5 OR and AND gate using a single neuron
- 6 Why XOR cannot be solved by one neuron
- 7 Multilayer neural network
- 8 Activation functions
- 9 Loss function
- 10 Backpropagation and gradient descent

Why Neural Networks?

Many real-world problems have complex nonlinear relationships.

Input features $\rightarrow$ Hidden patterns $\rightarrow$ Output decision

Examples:

- image classification,
- speech recognition,
- disease prediction,
- fraud detection,
- natural language processing.

Main Idea

A neural network learns useful intermediate representations from data.

Biological Inspiration

A biological neuron receives signals from other neurons, processes them, and sends an output signal. Artificial neural networks follow a simplified idea:

Inputs $\rightarrow$ Weighted sum $\rightarrow$ Activation $\rightarrow$ Output

Artificial Neuron

An artificial neuron computes a weighted combination of inputs and then applies a nonlinear activation function.

Artificial Neuron Model

Suppose a neuron receives inputs:

$$a_1, a_2, \dots, a_n$$

with weights:

$$w_{1j}, w_{2j}, \dots, w_{nj}$$

and bias:

$$w_{0j}$$

The input to neuron j is:

$$ln_j = w_{0j} + a_1 w_{1j} + a_2 w_{2j} + \dots + a_n w_{nj}$$

The output is:

$$a_j = g(ln_j)$$

where $g(\cdot)$ is the activation function.

Neuron in Vector Form

For compact notation:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

The neuron computes:

$$z = w^T x + b$$

$$a = g(z)$$

where:

z = linear score

a = activated output

Feedforward Neural Network

A feedforward neural network has connections only in the forward direction.

Input $\rightarrow$ Hidden Layer $\rightarrow$ Output

Definition

A feedforward network is a directed acyclic graph. There is no feedback loop.

Output depends on:

current input

and

current weights

It does not depend on previous hidden states.

Recurrent Neural Network

A recurrent neural network has feedback connections.

$$h_t = f(x_t, h_{t-1})$$

The output depends on:

current input

and

previous hidden state

Use

RNNs are useful for sequential data such as text, speech, and time series.

Past information → Current decision

Single Neuron as a Linear Classifier

For binary classification, a single neuron can be written as:

$$z = w_1x_1 + w_2x_2 + b$$

Prediction rule:

$$z \geq 0 \Rightarrow \hat{y} = 1$$

$$z < 0 \Rightarrow \hat{y} = 0$$

The decision boundary is:

$$w_1x_1 + w_2x_2 + b = 0$$

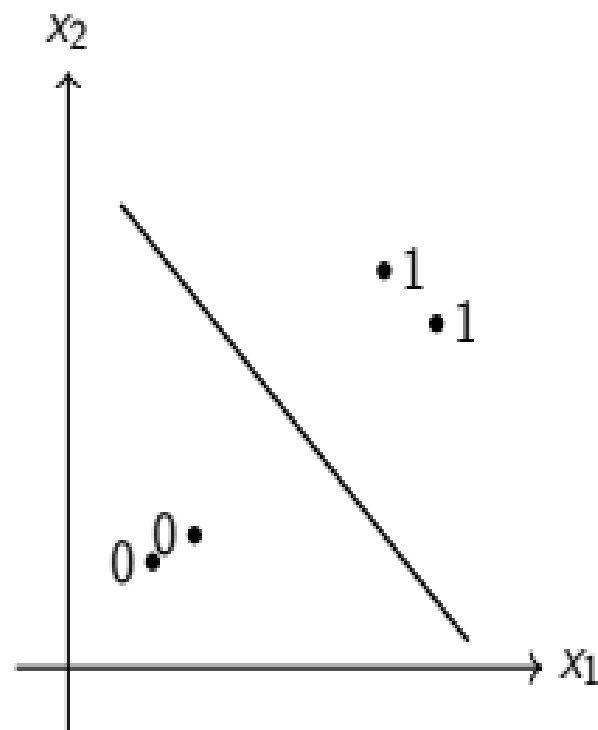
Key Point

A single neuron creates a linear decision boundary.

Linearly Separable Data

A dataset is linearly separable if one straight line can separate the two classes.

$$w_1x_1 + w_2x_2 + b = 0$$



A perceptron can solve this type of problem.

OR Gate Dataset

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Choose:

$$w_1 = 1, \quad w_2 = 1, \quad b = -0.5$$

Neuron score:

$$z = x_1 + x_2 - 0.5$$

Prediction rule:

$$z \geq 0 \Rightarrow 1, \quad z < 0 \Rightarrow 0$$

OR Gate Full Solution

$$z = x_1 + x_2 - 0.5$$

x_1	x_2	z	$\hat{y}$
0	0	$0 + 0 - 0.5 = -0.5$	0
0	1	$0 + 1 - 0.5 = 0.5$	1
1	0	$1 + 0 - 0.5 = 0.5$	1
1	1	$1 + 1 - 0.5 = 1.5$	1

Therefore:

$$w_1 = 1, \quad w_2 = 1, \quad b = -0.5$$

implements the OR gate.

AND Gate Full Solution

For AND gate, choose:

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.5$$

Then:

$$z = x_1 + x_2 - 1.5$$

x_1	x_2	z	$\hat{y}$
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	1

Changing the bias from -0.5 to -1.5 changes OR into AND.

Why XOR is Hard for One Neuron

XOR dataset:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Class 1 points:

$(0, 1), (1, 0)$

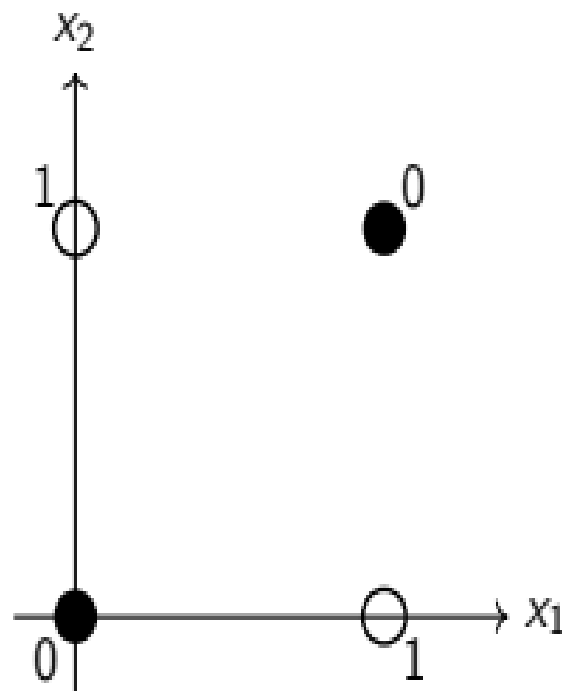
Class 0 points:

$(0, 0), (1, 1)$

Problem

No single straight line can separate the positive and negative XOR points.

XOR Geometric View



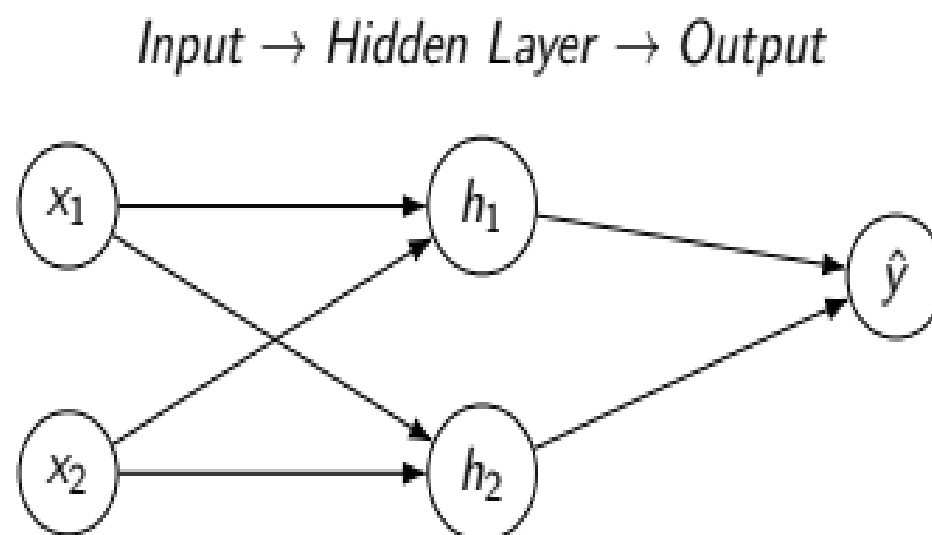
Reason

The positive examples lie diagonally opposite each other.

A single neuron gives one line, but XOR needs a nonlinear boundary.

Solution: Multilayer Neural Network

A multilayer neural network can solve XOR by combining multiple linear boundaries.



Hidden neurons create intermediate features.

XOR Using Hidden Neurons

XOR can be written as:

$$XOR = (x_1 \text{ OR } x_2) \text{ AND } NOT(x_1 \text{ AND } x_2)$$

Let:

$$h_1 = OR(x_1, x_2)$$

$$h_2 = AND(x_1, x_2)$$

Then:

$$XOR = h_1 \text{ AND } NOT(h_2)$$

This shows why a hidden layer helps.

Interpretation

The hidden layer transforms the original input into a new representation where XOR becomes easier.

Activation Functions

Common activation functions:

$$\text{Step}(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{ReLU}(z) = \max(0, z)$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Why Activation is Needed

Activation functions allow neural networks to learn nonlinear patterns.

Sigmoid Function

The sigmoid function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

$$0 < \sigma(z) < 1$$

$$\sigma(0) = 0.5$$

It is useful for binary classification:

$$\hat{y} = P(y = 1|x)$$

Derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Use

Sigmoid converts a real-valued score into probability.

ReLU Function

ReLU is:

$$\text{ReLU}(z) = \max(0, z)$$

So:

$$z < 0 \Rightarrow \text{ReLU}(z) = 0$$

$$z > 0 \Rightarrow \text{ReLU}(z) = z$$

Derivative:

$$\text{ReLU}'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

Why ReLU is Popular

ReLU is simple, fast, and helps deep networks train better than sigmoid in many cases.

Forward Pass in a Small Neural Network

Suppose we have two inputs, one hidden neuron, and one output neuron.

Hidden layer:

$$z_h = w_{h1}x_1 + w_{h2}x_2 + b_h$$

$$a_h = g(z_h)$$

Output layer:

$$z_o = w_o a_h + b_o$$

$$\hat{y} = \sigma(z_o)$$

Forward Pass

A forward pass computes predictions from inputs using current weights.

Forward Pass Example

Given:

$$x_1 = 2, \quad x_2 = 1$$

Hidden neuron:

$$w_{h1} = 0.4, \quad w_{h2} = 0.2, \quad b_h = -0.1$$

Output neuron:

$$w_o = 0.7, \quad b_o = 0.2$$

Use sigmoid activation for both hidden and output neurons.

Hidden score:

$$z_h = (0.4)(2) + (0.2)(1) - 0.1 = 0.9$$

Hidden activation:

$$a_h = \sigma(0.9) = \frac{1}{1 + e^{-0.9}} = 0.711$$

Output score:

$$z_o = (0.7)(0.711) + 0.2 = 0.698$$

Output prediction:

$$\hat{y} = \sigma(0.698) = 0.668$$

Loss Function

For binary classification, use binary cross-entropy:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

If:

$$y = 1$$

then:

$$L = -\log(\hat{y})$$

If:

$$y = 0$$

then:

$$L = -\log(1 - \hat{y})$$

Meaning

The loss is large when the model assigns low probability to the correct class.

Loss Example

From the previous example:

$$\hat{y} = 0.668$$

Suppose the true label is:

$$y = 1$$

Then:

$$L = -\log(0.668)$$

$$L = 0.403$$

If the true label were:

$$y = 0$$

then:

$$L = -\log(1 - 0.668)$$

$$L = -\log(0.332) = 1.102$$

Interpretation

The same prediction is good for $y = 1$, but bad for $y = 0$.

Gradient Descent

Gradient descent updates weights to reduce loss.

$$w := w - \eta \frac{\partial L}{\partial w}$$

$$b := b - \eta \frac{\partial L}{\partial b}$$

where:

η = learning rate

Meaning

Move parameters in the opposite direction of the gradient.

If the gradient is positive, the parameter decreases.

If the gradient is negative, the parameter increases.

Backpropagation: Network Setup

Consider a small neural network:

$x \rightarrow$ hidden neuron $\rightarrow$ output neuron

Forward equations:

$$z_h = w_h x + b_h$$

$$a_h = \sigma(z_h)$$

$$z_o = w_o a_h + b_o$$

$$\hat{y} = \sigma(z_o)$$

Loss:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Goal of Backpropagation

Backpropagation computes gradients:

$$\frac{\partial L}{\partial w_o}, \quad \frac{\partial L}{\partial b_o}, \quad \frac{\partial L}{\partial w_h}, \quad \frac{\partial L}{\partial b_h}$$

Then gradient descent updates:

$$w := w - \eta \frac{\partial L}{\partial w}$$

$$b := b - \eta \frac{\partial L}{\partial b}$$

Main idea:

Loss $\rightarrow$ Output neuron $\rightarrow$ Hidden neuron

Use chain rule from right to left.

Chain Rule Path for Output Weight

We want:

$$\frac{\partial L}{\partial w_o}$$

The path is:

$$w_o \rightarrow z_o \rightarrow \hat{y} \rightarrow L$$

So by chain rule:

$$\frac{\partial L}{\partial w_o} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_o} \cdot \frac{\partial z_o}{\partial w_o}$$

Since:

$$z_o = w_o a_h + b_o$$

$$\frac{\partial z_o}{\partial w_o} = a_h$$

Derivative of Binary Cross-Entropy

Binary cross-entropy:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Rewrite:

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Differentiate with respect to $\hat{y}$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}$$

Common denominator:

$$\frac{\partial L}{\partial \hat{y}} = \frac{-y(1 - \hat{y}) + (1 - y)\hat{y}}{\hat{y}(1 - \hat{y})}$$

$$= \frac{-y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1 - \hat{y})}$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

Derivative of Sigmoid

Sigmoid output:

$$\hat{y} = \sigma(z_o) = \frac{1}{1 + e^{-z_o}}$$

Rewrite:

$$\hat{y} = (1 + e^{-z_o})^{-1}$$

Differentiate:

$$\frac{\partial \hat{y}}{\partial z_o} = -(1 + e^{-z_o})^{-2}(-e^{-z_o})$$

$$\frac{\partial \hat{y}}{\partial z_o} = \frac{e^{-z_o}}{(1 + e^{-z_o})^2}$$

Since:

$$\hat{y} = \frac{1}{1 + e^{-z_o}}$$

and:

$$1 - \hat{y} = \frac{e^{-z_o}}{1 + e^{-z_o}}$$

Important Simplification

We have:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

and:

$$\frac{\partial \hat{y}}{\partial z_o} = \hat{y}(1 - \hat{y})$$

Now multiply:

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_o} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y})$$

Cancel:

$$\frac{\partial L}{\partial z_o} = \hat{y} - y$$

Define output error:

$$\delta_o = \hat{y} - y$$

$$\boxed{\delta_o = \hat{y} - y}$$

Output Layer Gradients

From chain rule:

$$\frac{\partial L}{\partial w_o} = \frac{\partial L}{\partial z_o} \cdot \frac{\partial z_o}{\partial w_o}$$

We know:

$$\frac{\partial L}{\partial z_o} = \delta_o = \hat{y} - y$$

and:

$$\frac{\partial z_o}{\partial w_o} = a_h$$

Therefore:

$$\frac{\partial L}{\partial w_o} = \delta_o a_h$$

For bias:

$$z_o = w_o a_h + b_o$$

$$\frac{\partial z_o}{\partial b_o} = 1$$

Chain Rule Path for Hidden Weight

Now we want:

$$\frac{\partial L}{\partial w_h}$$

The hidden weight affects the loss through this path:

$$w_h \rightarrow z_h \rightarrow a_h \rightarrow z_o \rightarrow \hat{y} \rightarrow L$$

So:

$$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_o} \cdot \frac{\partial z_o}{\partial a_h} \cdot \frac{\partial a_h}{\partial z_h} \cdot \frac{\partial z_h}{\partial w_h}$$

The first two terms are:

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_o} = \delta_o$$

Hidden Layer Components

We need the remaining terms.

Output neuron:

$$z_o = w_o a_h + b_o$$

So:

$$\frac{\partial z_o}{\partial a_h} = w_o$$

Hidden activation:

$$a_h = \sigma(z_h)$$

So:

$$\frac{\partial a_h}{\partial z_h} = \sigma'(z_h)$$

For sigmoid:

$$\sigma'(z_h) = a_h(1 - a_h)$$

Hidden score:

$$z_h = w_h x + b_h$$

So:

$$\frac{\partial z_h}{\partial w_h} = x$$

and:

$$\frac{\partial z_h}{\partial b_h} = 1$$

Hidden Weight Gradient

Combine all terms:

$$\frac{\partial L}{\partial w_h} = \delta_o \cdot w_o \cdot \sigma'(z_h) \cdot x$$

Since:

$$\sigma'(z_h) = a_h(1 - a_h)$$

$$\frac{\partial L}{\partial w_h} = \delta_o w_o a_h(1 - a_h)x$$

Define hidden error:

$$\delta_h = \delta_o w_o a_h(1 - a_h)$$

Then:

$$\frac{\partial L}{\partial w_h} = \delta_h x$$

For hidden bias:

$$\frac{\partial L}{\partial b_h} = \delta_h$$

Final Backpropagation Formulas

Forward pass:

$$z_h = w_h x + b_h, \quad a_h = \sigma(z_h)$$

$$z_o = w_o a_h + b_o, \quad \hat{y} = \sigma(z_o)$$

Output error:

$$\delta_o = \hat{y} - y$$

Output layer gradients:

$$\frac{\partial L}{\partial w_o} = \delta_o a_h$$

$$\frac{\partial L}{\partial b_o} = \delta_o$$

Hidden error:

$$\delta_h = \delta_o w_o a_h (1 - a_h)$$

Hidden layer gradients:

$$\frac{\partial L}{\partial w_h} = \delta_h x$$

Numerical Example: Forward Pass

Given:

$$x = 2, \quad y = 1$$

Hidden neuron:

$$w_h = 0.4, \quad b_h = -0.1$$

Output neuron:

$$w_o = 0.7, \quad b_o = 0.2$$

Hidden score:

$$z_h = w_h x + b_h = (0.4)(2) - 0.1 = 0.7$$

Hidden activation:

$$a_h = \sigma(0.7) = 0.668$$

Output score:

$$z_o = w_o a_h + b_o = (0.7)(0.668) + 0.2 = 0.668$$

Prediction:

$$o = \sigma(0.668) = 0.661$$

Numerical Example: Output Gradients

Output error:

$$\delta_o = \hat{y} - y$$

$$\delta_o = 0.661 - 1 = -0.339$$

Output weight gradient:

$$\frac{\partial L}{\partial w_o} = \delta_o a_h$$

$$\frac{\partial L}{\partial w_o} = (-0.339)(0.668)$$

$$\frac{\partial L}{\partial w_o} = -0.226$$

Output bias gradient:

$$\frac{\partial L}{\partial b_o} = \delta_o = -0.339$$

Numerical Example: Hidden Gradients

Hidden error:

$$\delta_h = \delta_o w_o a_h (1 - a_h)$$

$$\delta_h = (-0.339)(0.7)(0.668)(1 - 0.668)$$

$$\delta_h = (-0.339)(0.7)(0.668)(0.332)$$

$$\delta_h = -0.053$$

Hidden weight gradient:

$$\frac{\partial L}{\partial w_h} = \delta_h x$$

$$\frac{\partial L}{\partial w_h} = (-0.053)(2)$$

$$\frac{\partial L}{\partial w_h} = -0.106$$

Hidden bias gradient:

$$\frac{\partial L}{\partial b_h} = \delta_h$$

Numerical Example: Update Parameters

Use learning rate:

$$\eta = 0.1$$

Gradient descent rule:

$$w := w - \eta \frac{\partial L}{\partial w}$$

Output weight:

$$w_o := 0.7 - 0.1(-0.226) = 0.723$$

Output bias:

$$b_o := 0.2 - 0.1(-0.339) = 0.234$$

Hidden weight:

$$w_h := 0.4 - 0.1(-0.106) = 0.411$$

Hidden bias:

$$b_h := -0.1 - 0.1(-0.053) = -0.095$$

Since $y = 1$ and $\hat{y} = 0.661$, the model needs to increase the output probability.

Backpropagation Idea

Backpropagation computes gradients layer by layer using the chain rule.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Main Idea

Forward pass computes prediction. Backward pass computes how each weight contributed to the error.

Error at output → hidden layer gradients → input layer gradients

Backpropagation for Output Layer

For sigmoid output with binary cross-entropy:

$$\frac{\partial L}{\partial z_o} = \hat{y} - y$$

Let:

$$\delta_o = \hat{y} - y$$

Output weight gradient:

$$\frac{\partial L}{\partial w_o} = \delta_o a_h$$

Output bias gradient:

$$\frac{\partial L}{\partial b_o} = \delta_o$$

Using previous example:

$$\hat{y} = 0.668, \quad y = 1$$

$$\delta_o = 0.668 - 1 = -0.332$$

$$\frac{\partial L}{\partial w_o} = (-0.332)(0.711) = -0.236$$

$$\frac{\partial L}{\partial b_o} = -0.332$$

Backpropagation for Hidden Layer

Hidden error:

$$\delta_h = \delta_o w_o \sigma'(z_h)$$

Since:

$$\sigma'(z_h) = a_h(1 - a_h)$$

$$a_h = 0.711$$

$$\sigma'(z_h) = 0.711(1 - 0.711) = 0.205$$

$$\delta_h = (-0.332)(0.7)(0.205)$$

$$\delta_h = -0.0476$$

Hidden weight gradients:

$$\frac{\partial L}{\partial w_{h1}} = \delta_h x_1 = (-0.0476)(2) = -0.0952$$

$$\frac{\partial L}{\partial w_{h2}} = \delta_h x_2 = (-0.0476)(1) = -0.0476$$

One Gradient Descent Update

Let learning rate:

$$\eta = 0.1$$

Old parameters:

$$w_o = 0.7, \quad b_o = 0.2$$

$$w_{h1} = 0.4, \quad w_{h2} = 0.2, \quad b_h = -0.1$$

Output updates:

$$w_o := 0.7 - 0.1(-0.236) = 0.724$$

$$b_o := 0.2 - 0.1(-0.332) = 0.233$$

Hidden updates:

$$w_{h1} := 0.4 - 0.1(-0.0952) = 0.410$$

$$w_{h2} := 0.2 - 0.1(-0.0476) = 0.205$$

$$b_h := -0.1 - 0.1(-0.0476) = -0.095$$

Why Multilayer Networks Work

Each layer transforms the data into a new representation.

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \hat{y}$$

Early layers may learn simple features.

Middle layers may combine simple features.

Later layers may learn task-specific patterns.

Representation Learning

Neural networks learn features automatically from data instead of relying only on manually designed features.

Common Training Problems

❶ Overfitting:

training accuracy high, test accuracy low

❷ Vanishing gradient:

gradients become very small in early layers

❸ Exploding gradient:

gradients become too large

❹ Bad learning rate:

η too large or too small

Solutions

Use validation data, regularization, dropout, better initialization, and suitable activation functions.

Summary

- A neuron computes:

$$z = w^T x + b, \quad a = g(z)$$

- A single neuron is a linear classifier.
- OR and AND are linearly separable.
- XOR is not linearly separable.
- Multilayer networks solve nonlinear problems.
- Activation functions introduce nonlinearity.
- Backpropagation computes gradients using the chain rule.
- Gradient descent updates weights to reduce loss.